

Assorting Documentation for Integration into the ROS 2 Pipeline

Computer Vision Team Workflow, Independent Tasks and Cross-Team Integration

Prepared by: Tuan Thanh Lu (224259805)

Date: 26 July 2026

Purpose: document a practical CV-focused approach for integrating the selected ONNX models into ROS 2, identifying what the CV team can complete independently and where support from the simulation and planning subteams is required.

1. Document Purpose

This document organises the current integration discussion into one practical plan from the computer-vision team's perspective. The main CV objective is to create a reusable ROS 2 detection component that receives an image, runs the selected ONNX model, and publishes structured detections that can later be used by localisation and planning components.

The overall workflow is retained because the detector must eventually operate with Gazebo and the planning stack. However, the document separates the work that the CV team can develop and test on its own from the interface decisions and live integration activities that require collaboration with the other subteams.

2. Current Project Context

The CV team already has trained detection or segmentation models exported in ONNX format. The current leading deployment candidate is YOLO11n-seg. The ROS 2 node should still be configurable so another selected ONNX model can be loaded without a major source-code change.

The core CV task is to receive an image, apply the same preprocessing used during evaluation, run inference, decode the output, and publish classes, confidence scores, bounding boxes and masks. This core component can be developed before the final Gazebo scene or planning connection is ready.

CV team can complete independently	Requires coordination or input
------------------------------------	--------------------------------

Inspect ONNX model inputs, outputs, labels and export settings	Final deployment hardware and supported acceleration backend
Build and configure the ROS 2 detection node	ROS 2 distribution, workspace and launch conventions used by the project
Test with saved images, videos, mock publishers or rosbag data	Stable live RGB and camera-info topics from the simulation setup
Publish provisional 2D detections, masks and debug images	Exact message fields, topic names and update rate required by planning
Measure inference FPS, node latency and detection stability	End-to-end timing and robot-response testing with the integrated system

3. Model Format Decision

ONNX is suitable for the first ROS 2 integration. ROS 2 does not require the model to use a specific machine-learning format; the CV node only needs an inference backend that can load and execute the model. ONNX Runtime is a practical starting point because the same node can use CPU execution or a supported hardware-specific execution provider such as CUDA or TensorRT [1].

For this trimester, the CV team should keep the ONNX files as the main deployment artefacts and build the first working node using ONNX Runtime. If testing on the final robot computer shows that inference is too slow, an optimised backend such as TensorRT can be considered later. The CV interface should remain the same even when the backend changes.

Environment	Recommended format/backend	Reason
Development laptop or CPU-only system	ONNX + ONNX Runtime CPU	Simple and portable for initial testing
NVIDIA GPU workstation	ONNX + ONNX Runtime CUDA	Uses GPU acceleration without changing the model interface
NVIDIA Jetson or final embedded hardware	ONNX first; TensorRT if required	Optimise only after measuring the real bottleneck

4. Proposed ROS 2 Pipeline

CV-focused ROS 2 integration workflow

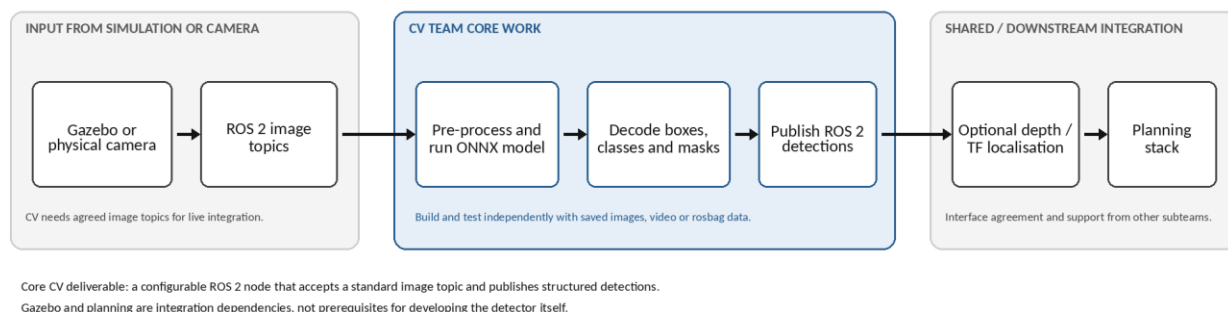


Figure 1. Overall pipeline with the CV-owned core and the main integration dependencies.

The main design principle is that the CV detector should not depend directly on Gazebo-specific code. It should subscribe to normal ROS 2 camera messages. This allows the CV team to test the same node using saved data, a mock image publisher, a Gazebo camera, or a physical camera without rewriting the inference component.

From the CV team's perspective, the workflow is:

1. Input dependency: a simulated or physical camera makes RGB images available as a ROS 2 image topic.
2. CV-owned: the detection node converts each image into the ONNX input format and runs inference.
3. CV-owned: post-processing produces classes, confidence scores, bounding boxes and instance masks where supported.
4. CV-owned: the node publishes structured detection messages and an optional annotated debug image.
5. Shared extension: detections may be combined with depth, camera calibration and TF when a 3D target is required.
6. Downstream dependency: the planning stack consumes the agreed target information and controls the robot.

5. Role of Gazebo

Gazebo is not used to convert or execute the ONNX model. For the CV team, Gazebo is mainly a source of simulated camera data and a place to test whether live detections behave correctly in the complete robotic environment. The CV team does not need to create the whole robot world before starting the detector node.

The simulation subteam would normally create the Gazebo world, robot and camera, then expose the RGB image, depth image and camera information through ROS 2, commonly using `ros_gz_bridge` [2]. Once those topic names and message types are provided, the CV team can connect its existing detector to the live stream and report any issues with encoding, image size, frame rate, lighting or object visibility.

The YOLOv5-in-Gazebo video shared by the project leader is useful as an architectural reference. The CV team can retain the idea of subscribing to a simulated camera and publishing detections, while replacing the YOLOv5-specific model code with the team's configurable YOLO11n or YOLO11n-seg ONNX implementation.

Key point: the CV team can build and validate the detector independently using standard ROS 2 image topics. Collaboration with the simulation team is needed when connecting to the real Gazebo camera topics and testing the complete simulated scene.

6. ROS 2 Detection Node Design

The ROS 2 detection node is the main CV-owned software component. The ONNX model should be loaded once when the node starts, and model selection should be controlled through ROS 2 parameters. This lets the CV team test different selected models without changing the external topic interface.

6.1 Main processing stages

- Subscribe to the RGB image topic using `sensor_msgs/msg/Image`.
- Convert the ROS image to an OpenCV-compatible array, normally through `cv_bridge`.
- Apply the same colour conversion, letterbox resize, normalisation and tensor layout used during model evaluation.
- Run the ONNX model through the selected ONNX Runtime execution provider.
- Decode the output, apply confidence filtering and non-maximum suppression where required.
- Map bounding boxes and masks back to the original image resolution.
- Publish structured detections and an optional debug image.

6.2 Suggested parameters

```
model_path: "/models/yolo11n_seg.onnx"
labels_path: "/models/classes.yaml"
input_width: 640
input_height: 640
confidence_threshold: 0.40
nms_threshold: 0.45
execution_provider: "CPU"
publish_debug_image: true
```

The CV team should inspect the exact ONNX input and output shapes before finalising post-processing. Different export settings may change whether non-maximum suppression is included and how segmentation masks are represented. This can be verified independently by comparing the ROS 2 node with the existing evaluation notebook on the same test images.

7. ROS 2 Topics and Message Interfaces

The CV team can define and test provisional input and output topics early, but the final interface should be agreed with the simulation and planning subteams. The CV output should be machine-readable rather than only an annotated image. The ROS vision_msgs package provides standard messages for computer-vision pipelines, including 2D and 3D detections [3].

Example topic	Publisher / owner	Subscriber	CV relevance
/camera/color/image_raw	Simulation or camera team	CV detector node	Required RGB input; CV can mock this topic during development
/camera/depth/image_raw	Simulation or camera team	CV/shared localisation	Needed only if 3D localisation is included
/camera/camera_info	Simulation or camera team	CV/shared localisation	Provides camera intrinsics for 3D conversion
/detector/detections_2d	CV team	Localisation or planning	Core structured CV output
/detector/masks	CV team	Localisation or visualisation	Optional segmentation output from YOLO11n-seg
/detector/debug_image	CV team	RViz or developers	CV-owned visual check; not the main planning interface
/detector/target_pose	CV or shared localisation	Planning stack	Collaborative output only if 3D target generation is assigned to CV

These topic names are examples. The CV team can begin with configurable topic parameters, then update the launch file after the shared naming convention is confirmed. The original image timestamp and camera frame ID should be preserved so later depth and TF processing refers to the same observation.

8. From 2D Detection to Planning Information

The core CV deliverable can reasonably end at 2D detections and segmentation masks if that is the agreed interface. A robotic extraction planner may instead require a 3D point or pose, but ownership of that localisation stage should be confirmed rather than automatically assigned to the CV team.

If the CV team is asked to prototype or support 3D localisation and RGB-D data is available, a practical process is:

1. Select valid depth pixels inside the predicted mask or bounding box.
2. Reject missing depth values and obvious outliers.
3. Use the camera intrinsics to convert the selected image point into a 3D point in the camera frame.
4. Use tf2 to transform the point from the camera frame into the robot base or workcell frame.

5. Publish a target point, target pose or Detection3D message for the planning stack.

For battery extraction, the segmentation mask may give a more stable object centre than the bounding-box centre, especially when the object is rotated or partly covered. However, depth quality, camera calibration, TF frames and the required grasp representation depend on the simulation and planning setup. These items should be treated as collaborative work unless the project explicitly allocates them to the CV team.

9. CV Team Work Breakdown and Collaboration Boundaries

Work area	CV team can do independently	Collaboration or dependency
Model preparation	Validate ONNX loading, tensor shapes, label mapping and preprocessing.	Confirm final deployment hardware and available execution providers.
ROS 2 detector node	Implement parameters, image subscription, inference, post-processing and error handling.	Follow the project workspace, package and launch conventions.
Input testing	Use test images, video, a mock image publisher or rosbag data.	Simulation provides stable live Gazebo RGB/depth topics and camera settings.
Detection output	Publish provisional Detection2DArray, masks and debug images.	Planning confirms required fields, topic names, update rate and failure behaviour.
3D localisation	Prototype mask/depth processing when suitable data is available.	Needs camera_info, TF frames and the target coordinate frame expected by planning.
Gazebo validation	Launch the CV node, inspect detections and measure inference behaviour.	Simulation owns the world, camera, bridge and scene adjustments.
Planning integration	Publish the agreed message and help trace CV-side issues.	Planning subscribes to the output and controls target selection and robot action.

The CV team should not need to wait for every other component before making progress. It can complete the model backend, ROS 2 node, post-processing, provisional messages and most unit testing with local or recorded inputs. Collaboration becomes essential at the interface checkpoints: receiving the actual Gazebo topics, agreeing on planning outputs, aligning coordinate frames and testing the full pipeline.

10. Recommended Implementation Sequence

1. Collaboration checkpoint: confirm the project's ROS 2 distribution, workspace structure and package conventions.
2. CV independent work: inspect one selected ONNX model and record its input/output names, tensor shapes, class labels and preprocessing requirements.
3. CV independent work: build a small ROS 2 node that loads the model once and processes a configurable image topic.
4. CV independent work: implement post-processing and publish 2D detections, masks where applicable, and a debug image.
5. CV independent work: compare the ROS 2 output with the existing evaluation notebook using the same images.
6. Collaboration checkpoint: agree with the simulation team on RGB, depth and camera-info topic names, encodings and frame IDs.
7. Joint integration: connect the detector to the Gazebo camera stream and document any domain, resolution or frame-rate issues.
8. Collaboration checkpoint: agree with planning on the required message, target frame, update rate and behaviour when no object is detected.
9. Shared or optional work: add depth-based localisation and TF conversion only when it is required and ownership is confirmed.
10. Joint integration: connect the agreed CV output to the planning stack and test the simulated end-to-end response.

- CV-led measurement: benchmark inference FPS and node latency, then measure end-to-end latency with the other subteams before considering CUDA or TensorRT optimisation.

11. Testing and Acceptance Criteria

Testing should begin with CV-owned checks and then move to shared integration checks. This avoids waiting for the complete robot simulation before verifying the detector itself.

- CV-owned: the model loads once and the node remains stable during continuous image streaming.
- CV-owned: the ROS 2 node produces the same classes and similar confidence values as the existing evaluation code.
- CV-owned: bounding boxes and masks align correctly with the original image after resizing and letterboxing.
- CV-owned: the node can switch between selected ONNX models through configuration.
- CV-owned: empty detections, invalid images and temporary input loss are handled without crashing.
- Joint check: Gazebo images reach the CV node with the agreed encoding, dimensions, timestamp and frame ID.
- Joint check: the localisation or planning component can consume the published message without manual conversion.
- CV-led measurement: inference FPS and image-to-detection latency are recorded on the available hardware.
- Joint measurement: end-to-end latency is recorded from simulated image capture to the planning target or robot response.

12. Main Risks and Mitigation

Risk	Likely effect	Mitigation
Different preprocessing from training/evaluation	Incorrect or unstable detections	Reuse and test the exact resize, colour and normalisation logic.
ONNX output shape differs from the expected decoder	Post-processing failure	Inspect model metadata and validate one export at a time.
Gazebo topics or scenes are not ready	CV progress is blocked unnecessarily	Use saved images, rosbag data or a mock image publisher until live topics are available.
Topic or message mismatch between teams	Integration delays	Maintain a short shared interface agreement with examples.
2D detections are insufficient for planning	No usable robot target	Confirm the required target representation early and assign ownership of depth/TF work.
Inference is too slow on deployment hardware	Low FPS or delayed planning	Measure first, then test CUDA, TensorRT, smaller inputs or controlled frame skipping.

13. Suggested ROS 2 Package Structure

```
battery_vision_bringup/  
  launch/  
    gazebo_detection.launch.py  
  config/  
    detector.yaml  
    bridge.yaml  
  
battery_detector/  
  detector_node.py  
  model_backend.py  
  preprocessing.py  
  postprocessing.py
```

```
battery_localisation/  
depth_localisation_node.py  
tf_transform.py
```

```
battery_detection_msgs/ # only if standard messages are insufficient  
msg/  
BatteryDetection.msg
```

The detector package is the core CV deliverable. The localisation package should be included only if 3D localisation is assigned to the CV team, while the bringup package can contain shared launch and configuration files used during Gazebo integration. This structure lets the CV team improve the model backend without changing the planning interface.

14. Expected CV Team Deliverables

- A short ONNX model compatibility note covering input/output tensors, labels, preprocessing and selected runtime backend.
- A configurable ROS 2 detector node that loads the selected ONNX model and accepts a standard image topic.
- Validated preprocessing and post-processing that reproduce the existing evaluation results on shared test images.
- Structured detection and mask topics, together with an annotated debug-image topic for inspection.
- A CV launch/configuration file that works with a generic camera topic and can be mapped to Gazebo topics once provided.
- A concise integration and test record containing screenshots, example topic outputs, FPS, node latency and the agreed hand-off requirements for simulation and planning.

15. Conclusion

The current ONNX models are appropriate for the first ROS 2 integration and do not need to be converted immediately. The main CV responsibility is to turn the selected model into a reliable, configurable ROS 2 detection node that accepts a standard image stream and publishes consistent machine-readable results.

The CV team can complete most of this work independently using saved or recorded inputs. Gazebo becomes necessary for live simulated-camera testing, while planning collaboration is necessary to finalise the output message and any 3D target requirements. Keeping these boundaries clear should allow the CV team to progress without waiting unnecessarily, while still producing a component that fits the complete project pipeline.

References and Useful Starting Points

[\[1\] ONNX Runtime: Execution Providers](#)

[\[2\] Gazebo: Use ROS 2 to interact with Gazebo](#)

[\[3\] ROS 2 vision_msgs package documentation](#)

[\[4\] Ultralytics YOLO11 model documentation](#)

[\[5\] Ultralytics model export documentation](#)

[\[6\] YOLOv5 in Gazebo reference video shared by Moksh](#)